REVISION CHEAT SHEET:

DESIGN PATTERNS

Design Patterns in C# and .NET

Interview-ready notes · examples · C# · diagrams · clickable index

INDEX / TABLE OF CONTENTS

Click a section name to jump to that section.

1. What Is a Design Pattern?
2. Pattern vs Algorithm
3. Pattern vs Design Principle
4. Pattern vs Architecture
5. Why Patterns Are Useful
6. The Biggest Design-Pattern Mistake
7. GoF Design Pattern Categories
8. Creational Patterns
9. Structural Patterns
10. Behavioral Patterns
PART 1 — CREATIONAL PATTERNS
11. Singleton Pattern
Definition
12. Traditional Singleton Flow
13. Singleton in Modern .NET
14. When Singleton Is Appropriate
15. When Singleton Is Dangerous
16. Singleton Advantages
17. Singleton Disadvantages
18. Singleton Rule
19. Factory Method Pattern
Problem
20. Factory Method Idea
21. Practical C# Factory
22. When to Use Factory
23. Do Not Create Factories for Everything
24. Factory vs DI Container
25. Abstract Factory Pattern
26. Abstract Factory Example
27. Abstract Factory Mental Model
28. Factory Method vs Abstract Factory
29. Builder Pattern
30. Bad Constructor Explosion
31. Builder Example
32. Builder Advantages
33. Builder Disadvantages
34. Builder vs Object Initializer
35. Builder-Like .NET APIs
36. Prototype Pattern
37. Shallow vs Deep Copy
38. When Prototype Helps
PART 2 — STRUCTURAL PATTERNS
39. Adapter Pattern
40. Adapter Example
41. Adapter Flow
42. Adapter Advantages
43. Adapter vs Facade
44. Bridge Pattern
45. Bridge Example
46. Composite Pattern
47. Fundraising Composite Example
48. Decorator Pattern
49. Receipt Decorator Example
50. Multiple Decorators
51. Decorator vs Inheritance
52. Decorator in HttpClient
53. Facade Pattern
54. Facade Example
55. Facade vs Service Layer
56. Flyweight Pattern
57. Flyweight Concept
58. Flyweight Example
59. When Flyweight Is Useful
60. Proxy Pattern
61. Proxy Example
62. Proxy vs Decorator

PART 3 — BEHAVIORAL PATTERNS
63. Strategy Pattern
64. Bad Strategy Problem
65. Strategy Implementation
66. Strategy Resolver
67. Strategy with Keyed DI
68. Strategy with Delegates
69. Strategy vs Factory
70. Observer Pattern
71. Observer in .NET
72. Simple Event Observer Example
73. Observer Advantages
74. Observer Risks
75. Command Pattern
76. Why Command Is Useful
77. Command vs CQRS
78. Chain of Responsibility Pattern
79. Validation Chain Example
80. Chain Flow
81. ASP.NET Core Middleware and Chain of Responsibility
82. Template Method Pattern
83. Template Method Example
84. Template Method vs Strategy
85. State Pattern
86. State Pattern Example
87. State Pattern Flow
88. When State Pattern Is Worth It
89. Mediator Pattern
90. Simple Mediator Example
91. Mediator Advantages
92. Mediator Risks
93. Iterator Pattern
94. Iterator Example
95. Iterator Benefits
96. Memento Pattern
97. Memento Example
98. Memento Caveat
99. Visitor Pattern
100. Visitor Strength
101. Visitor Weakness
102. Interpreter Pattern
103. Simplified Interpreter
104. Interpreter Warning
PART 4 — PATTERNS FREQUENTLY SEEN IN .NET BACKENDS
105. Repository Pattern
106. Unit of Work Pattern
107. Service Layer Pattern
108. Specification Pattern
109. Options Pattern
110. Middleware Pipeline Pattern
111. CQRS Pattern
112. Outbox Pattern — Boundary Note
PART 5 — PATTERN COMPARISONS
113. Factory vs Builder
114. Factory vs Strategy
115. Adapter vs Decorator
116. Adapter vs Facade
117. Decorator vs Proxy
118. Strategy vs State
119. Strategy vs Template Method
120. Observer vs Mediator
121. Chain of Responsibility vs Decorator
122. Command vs Strategy
123. Pattern Selection Table
PART 6 — ASP.NET CORE PATTERN MAPPING
124. Common .NET Pattern Relationships
125. Patterns and DI Work Together
PART 7 — FUNDRAISING PLATFORM END-TO-END EXAMPLE
126. Requirement
127. Validation — Chain of Responsibility
128. Payment Selection — Strategy
129. Vendor Integration — Adapter
130. Receipt — Decorator
131. Gift State — State Pattern Only If Needed
132. Completion Notification — Observer
133. Application Boundary — Facade / Service Layer
134. Persistence — Repository / Unit of Work
135. Combined Flow
PART 8 — PERFORMANCE CONSIDERATIONS
136. Patterns Have Costs
137. Strategy Performance
138. Decorator Depth
139. Flyweight and Pooling
PART 9 — MEMORY CONSIDERATIONS

140. Singleton Lifetime
141. Observer Lifetime
142. Decorator Object Graph
PART 10 — THREAD-SAFETY CONSIDERATIONS
143. Pattern Does Not Guarantee Thread Safety
144. Singleton Special Rule
145. Observer Concurrency
PART 11 — SECURITY CONSIDERATIONS
146. Adapter Security Boundary
147. Proxy and Authorization
148. Strategy and Tenant Isolation
149. Singleton Security Risk
150. Decorator Security
PART 12 — COMMON PATTERN MISTAKES
151. Pattern Fever
152. Factory for Every Constructor
153. Singleton for Convenience
154. Strategy for One Algorithm
155. Decorator for Core Business Logic
156. Observer for Mandatory Workflow
157. State Pattern Too Early
158. Abstract Factory Too Early
159. Mediator Everywhere
160. Repository over EF for No Benefit
161. Inheritance Instead of Strategy
162. Patterns Without Tests
PART 13 — BEST PRACTICES
163. Design From the Problem
164. Prefer Composition
165. Keep Interfaces Small and Meaningful
166. Use Framework Features
167. Use Pattern Names for Communication
168. But Document Intent
PART 14 — PRACTICAL EXERCISES
Exercise 1 — Strategy
Exercise 2 — Factory
Exercise 3 — Adapter
Exercise 4 — Decorator
Exercise 5 — Chain of Responsibility
Exercise 6 — Observer
Exercise 7 — Builder
Exercise 8 — State
Exercise 9 — Facade
Exercise 10 — Proxy
Exercise 11 — Composite
Exercise 12 — Pattern Selection
PART 15 — EXACTLY 25 INTERVIEW QUESTIONS
1. What is a design pattern?
2. What are the three categories of GoF patterns?
3. What is the Singleton pattern and what is its main risk?
4. What is Factory Method?
5. What is the difference between Factory Method and Abstract Factory?
6. What is the Builder pattern?
7. What is the Prototype pattern?
8. What is the Adapter pattern?
9. What is the Decorator pattern?
10. What is the difference between Decorator and Proxy?
11. What is the Facade pattern?
12. What is the Composite pattern?
13. What is the Strategy pattern?
14. What is the difference between Strategy and State?
15. What is the Observer pattern?
16. What is the Command pattern?
17. What is Chain of Responsibility?
18. What is the Template Method pattern?
19. What is the difference between Template Method and Strategy?
20. What is the Mediator pattern?
21. What is the Iterator pattern and how does C# support it?
22. When should you use State instead of a simple enum?
23. How are Repository and Unit of Work related to EF Core?
24. How do you decide whether to use a design pattern?
25. Which patterns would you commonly use in the Fundraising Platform?
PART 16 — QUICK REVISION
PART 17 — FINAL PATTERN-SELECTION FLOW
PART 18 — FINAL END-TO-END CONCEPT FLOW

1. What Is a Design Pattern?

A design pattern is a:

Reusable design idea for solving a recurring software-design proble

DESIGN PATTERNS

A pattern is **not** usually a finished library or a piece of code that you copy unchanged.

Instead, it describes:

EXAMPLE / FLOW

```
Problem
    ↓
Context
    ↓
Forces / trade-offs
    ↓
Common structure
    ↓
Responsibilities
    ↓
Solution idea
```

For example:

EXAMPLE / FLOW

```
Problem:
Several payment methods calculate fees differently.

Possible pattern:
Strategy

Idea:
Define one common contract and move each
fee algorithm into an interchangeable implementation.
```

2. Pattern vs Algorithm

An algorithm describes:

EXAMPLE / FLOW

```
How to calculate something step by step
```

Example:

EXAMPLE / FLOW

```
Binary search
Merge sort
Dijkstra's algorithm
```

A design pattern describes:

EXAMPLE / FLOW

```
How objects/classes/components
should collaborate
```

Example:

EXAMPLE / FLOW

```
Factory
Strategy
Decorator
```

```
Observer
Adapter
```

3. Pattern vs Design Principle

Design principle:

EXAMPLE / FLOW

```
General guidance
```

Examples:

EXAMPLE / FLOW

```
SOLID
DRY
KISS
YAGNI
Composition over inheritance
```

Design pattern:

EXAMPLE / FLOW

```
More concrete recurring structural solution
```

Example:

EXAMPLE / FLOW

```
OCP says:
Design for meaningful extension.

Strategy provides:
One possible structural technique
for implementing that extension.
```

Therefore:

EXAMPLE / FLOW

```
Principle
    →
"Why should the design behave this way?"

Pattern
    →
"How might we structure the objects?"
```

4. Pattern vs Architecture

Architecture operates at a larger scale.

Examples:

EXAMPLE / FLOW

```
Layered Architecture
Clean Architecture
Hexagonal Architecture
Microservices
Event-driven architecture
CQRS architecture
```

Design patterns usually operate at a smaller level:

DESIGN PATTERNS

EXAMPLE / FLOW

```
Object creation
Object collaboration
Behavior selection
Interface adaptation
Notification
Processing pipelines
```

You can use:

EXAMPLE / FLOW

```
Strategy
Decorator
Factory
Repository
```

inside a larger:

EXAMPLE / FLOW

```
Clean Architecture
```

application.

5. Why Patterns Are Useful

Patterns provide a shared design vocabulary.

Instead of explaining:

> “I created one interface, several algorithms, and choose the implementation at runtime.”

you can say:

EXAMPLE / FLOW

```
"We're using Strategy."
```

Experienced developers immediately understand much of the design.

Patterns also help with:

EXAMPLE / FLOW

```
Separation of concerns

Extensibility

Testability

Reusability

Encapsulation

Reducing coupling

Communicating architecture
```

6. The Biggest Design-Pattern Mistake

The biggest mistake is:

EXAMPLE / FLOW

```
"I learned 23 patterns.

Now every class must use one."
```

That creates:

EXAMPLE / FLOW

```
Pattern-driven development
instead of
requirement-driven design
```

Use a pattern because:

EXAMPLE / FLOW

```
a recurring design problem exists
```

not because:

EXAMPLE / FLOW

```
the pattern is fashionable.
```

7. GoF Design Pattern Categories

The classic **Gang of Four (GoF)** catalog contains 23 patterns.

They are grouped into:

EXAMPLE / FLOW

```
Creational
Structural
Behavioral
```

8. Creational Patterns

Creational patterns focus on:

EXAMPLE / FLOW

```
How objects are created
```

The five classic patterns are:

EXAMPLE / FLOW

```
1. Singleton
2. Factory Method
3. Abstract Factory
4. Builder
5. Prototype
```

9. Structural Patterns

Structural patterns focus on:

EXAMPLE / FLOW

```
How objects/classes are connected
```


The seven patterns are:

EXAMPLE / FLOW

```
6. Adapter
7. Bridge
8. Composite
9. Decorator
10. Facade
11. Flyweight
12. Proxy
```

10. Behavioral Patterns

Behavioral patterns focus on:

EXAMPLE / FLOW

```
How objects communicate
and how behavior is selected
```

The eleven patterns are:

EXAMPLE / FLOW

```
13. Chain of Responsibility
14. Command
15. Interpreter
16. Iterator
17. Mediator
18. Memento
19. Observer
20. State
21. Strategy
22. Template Method
23. Visitor
```

We will cover all of them.

PART 1 — CREATIONAL PATTERNS

11. Singleton Pattern

Definition

Singleton ensures that:

EXAMPLE / FLOW

```
Only one logical instance
of a component exists
within a particular lifetime boundary
```

and provides shared access to it.

Traditional implementation:

C#

```
public sealed class SystemClock
{
```

```
    private static readonly SystemClock _instance = new();

    private SystemClock()
    {
    }

    public static SystemClock Instance =>
        _instance;

    public DateTimeOffset UtcNow =>
        DateTimeOffset.UtcNow;
}
```

Usage:

C#
```
DateTimeOffset now = SystemClock.Instance.UtcNow;
```

12. Traditional Singleton Flow

EXAMPLE / FLOW
```
Caller A ─┐
Caller B ─┼─→ Singleton.Instance
Caller C ─┘
              ↓
           One Object
```

13. Singleton in Modern .NET

In ASP.NET Core, you usually **do not manually implement Singleton using static fields**.

Instead:

C#
```
builder.Services
    .AddSingleton<
        IClock,
        SystemClock>();
```

Then:

C#
```
public GiftService(
    IClock clock)
{
    _clock = clock;
}
```

This gives one instance for the root DI container lifetime while preserving:

EXAMPLE / FLOW
```
DI
Testability
Explicit dependencies
Container-managed disposal
```

14. When Singleton Is Appropriate

Good candidates:

EXAMPLE / FLOW

```
Stateless shared services

Thread-safe metadata providers

Immutable configuration snapshots

Process-wide caches with explicit limits

Shared expensive resources designed for reuse
```

15. When Singleton Is Dangerous

Avoid singleton for:

EXAMPLE / FLOW

```
Current user state

Tenant state

Request data

DbContext

Mutable non-thread-safe collections

Per-request authorization data
```

Why?

EXAMPLE / FLOW

```
Singleton
    ↓
shared by all requests
```

Therefore singleton mutable state can cause:

EXAMPLE / FLOW

```
Race conditions
Cross-request contamination
Cross-tenant exposure
Memory retention
```

16. Singleton Advantages

EXAMPLE / FLOW

```
Avoid repeated construction

Useful for truly global/shared state

Easy shared caching

Can reduce expensive initialization
```

17. Singleton Disadvantages

EXAMPLE / FLOW

```
Global lifetime

Thread-safety requirements

Can hide shared mutable state

Harder tests if implemented statically

Can retain huge object graphs

Can become service-locator-like global state
```

18. Singleton Rule

Prefer:

EXAMPLE / FLOW

```
DI-managed singleton
```

over:

EXAMPLE / FLOW

```
static Singleton.Instance
```

unless a static singleton is truly appropriate.

19. Factory Method Pattern

Problem

Suppose we need several receipt types:

EXAMPLE / FLOW

```
Email receipt
Printed receipt
SMS receipt
```

Bad:

C#

```
public IReceiptSender Create(
    ReceiptType type)
{
    if (
        type ==
        ReceiptType.Email)
    {
        return new EmailReceiptSender();
    }

    if (
        type ==
        ReceiptType.Print)
    {
        return new PrintReceiptSender();
    }
```

```
    throw new
        NotSupportedException();
}
```


Construction logic is scattered.

20. Factory Method Idea

Factory Method encapsulates:

EXAMPLE / FLOW

```
Object creation decision
```

behind a method or creator abstraction.

Example:

C#

```
public interface IReceiptSender
{
    Task SendAsync(
        Gift gift,
        CancellationToken cancellationToken);
}
```

Creator:

C#

```
public abstract class ReceiptSenderCreator
{
    public abstract IReceiptSender
        CreateSender();
}
```

Concrete creator:

C#

```
public sealed class
    EmailReceiptSenderCreator
    :
    ReceiptSenderCreator
{
    public override IReceiptSender
        CreateSender()
    {
        return new
            EmailReceiptSender();
    }
}
```

21. Practical C# Factory

In real ASP.NET Core code, a simpler factory object is often enough:

C#

```
public interface IReceiptSenderFactory
{
    IReceiptSender Create(
```

```
        ReceiptType type);
}
```

C#

```
public sealed class ReceiptSenderFactory :
    IReceiptSenderFactory
{
    private readonly
        IEnumerable<IReceiptSender>
        _senders;

    public ReceiptSenderFactory(
        IEnumerable<IReceiptSender> senders)
    {
        _senders = senders;
    }

    public IReceiptSender Create(
        ReceiptType type)
    {
        return _senders.Single(
            sender =>
                sender.CanHandle(
                    type));
    }
}
```

22. When to Use Factory

Use Factory when:

EXAMPLE / FLOW

```
Construction is complicated

Caller should not know concrete type

Implementation depends on runtime input

Creation rules may change

Construction itself requires dependencies
```

23. Do Not Create Factories for Everything

Bad:

EXAMPLE / FLOW

```
GiftServiceFactory
GiftRepositoryFactory
GiftValidatorFactory
GiftMapperFactory
```

when DI can simply inject the service.

If:

C#

```
public GiftService(
    IGiftRepository repository)
```

works naturally, a factory adds no value.

24. Factory vs DI Container

DI container:

EXAMPLE / FLOW

```
Builds general application object graphs
```

Factory:

EXAMPLE / FLOW

```
Handles a meaningful creation decision
inside application behavior
```

Do not replace every factory with:

C#

```
IServiceProvider.GetRequiredService(...)
```

because that creates service locator.

Microsoft's current DI guidance explicitly recommends avoiding service-locator style runtime resolution when constructor injection can express dependencies directly.

25. Abstract Factory Pattern

Abstract Factory creates:

> **Families of related objects without exposing their concrete classes.**

Imagine payment integrations.

Stripe family:

EXAMPLE / FLOW

```
StripePaymentGateway
StripeRefundGateway
StripeTokenizationService
```

Another provider:

EXAMPLE / FLOW

```
AdyenPaymentGateway
AdyenRefundGateway
AdyenTokenizationService
```

26. Abstract Factory Example

C#

```
public interface IPaymentIntegrationFactory
{
    IPaymentGateway
        CreatePaymentGateway();

    IRefundGateway
```

```
        CreateRefundGateway();
}
```

Stripe:

```C#
public sealed class
    StripeIntegrationFactory
    :
    IPaymentIntegrationFactory
{
    public IPaymentGateway
        CreatePaymentGateway()
        =>
        new StripePaymentGateway();

    public IRefundGateway
        CreateRefundGateway()
        =>
        new StripeRefundGateway();
}
```

27. Abstract Factory Mental Model

EXAMPLE / FLOW

```
Client
   ↓
IPaymentIntegrationFactory
       ↓
 ┌─────────────────┐
 ↓                 ↓
Payment          Refund
Gateway          Gateway
```

Switch factory:

EXAMPLE / FLOW

```
Stripe Factory
    →
Stripe product family

Adyen Factory
    →
Adyen product family
```

28. Factory Method vs Abstract Factory

Factory Method:

EXAMPLE / FLOW

```
Creates one kind of product
```

Abstract Factory:

EXAMPLE / FLOW

```
Creates a family
of related products
```

29. Builder Pattern

Builder separates:

EXAMPLE / FLOW

```
Complex object construction
```

from:

EXAMPLE / FLOW

```
Final object representation
```

Useful when an object requires many optional or staged inputs.

30. Bad Constructor Explosion

C#

```
Gift(
    tenantId,
    constituentId,
    amount,
    campaignId,
    fundId,
    paymentMethod,
    externalReference,
    anonymous,
    tributeName,
    comments,
    receiptEmail,
    sourceCode,
    batchId);
```

Difficult to understand.

31. Builder Example

C#

```
public sealed class GiftDraftBuilder
{
    private long _tenantId;
    private long _constituentId;
    private decimal _amount;
    private long? _campaignId;
    private string? _comments;

    public GiftDraftBuilder
        ForTenant(
            long tenantId)
    {
        _tenantId = tenantId;

        return this;
    }

    public GiftDraftBuilder
        ForConstituent(
            long constituentId)
    {
        _constituentId = constituentId;
```

```
        return this;
    }

    public GiftDraftBuilder
        WithAmount(
            decimal amount)
    {
        _amount = amount;

        return this;
    }

    public GiftDraftBuilder
        ForCampaign(
            long campaignId)
    {
        _campaignId = campaignId;

        return this;
    }

    public Gift Build()
    {
        return Gift.Create(
            _tenantId,
            _constituentId,
            _amount,
            _campaignId);
    }
}
```

Usage:

C#

```
Gift gift =
    new GiftDraftBuilder()
        .ForTenant(10)
        .ForConstituent(200)
        .WithAmount(500m)
        .ForCampaign(30)
        .Build();
```

32. Builder Advantages

EXAMPLE / FLOW

```
Readable staged construction

Handles many optional parameters

Can validate before Build

Can create multiple representations

Useful in tests
```

33. Builder Disadvantages

EXAMPLE / FLOW

```
Extra class
```

```
Extra mutable construction state
*
Overkill for simple objects

Can duplicate domain validation
```

34. Builder vs Object Initializer

Simple DTO:

```C#
CreateGiftRequest request =
    new()
    {
        ConstituentId =
            10,

        Amount =
            100m
    };
```

No Builder required.

Use Builder when:

```
EXAMPLE / FLOW
construction has meaningful complexity
```

not simply because:

```
EXAMPLE / FLOW
object has properties.
```

35. Builder-Like .NET APIs

Modern .NET APIs frequently use a fluent builder style:

```C#
WebApplicationBuilder builder =
    WebApplication
        .CreateBuilder(args);

builder.Services
    .AddControllers();
```

This is builder-oriented API design, although API names containing `Builder` should not automatically be assumed to be textbook GoF Builder implementations.

36. Prototype Pattern

Prototype creates new objects by:

```
EXAMPLE / FLOW
copying an existing prototype
```

rather than constructing completely from scratch.

Example:

```C#
public sealed record GiftTemplate(
    long CampaignId,
    long FundId,
    string SourceCode);
```

Prototype:

```C#
GiftTemplate annual =
    new(
        CampaignId:
            100,
        FundId:
            200,
        SourceCode:
            "ANNUAL");
```

Create variation:

```C#
GiftTemplate majorGift =
    annual with
    {
        SourceCode =
            "MAJOR"
    };
```

Records make copy-with-modification especially convenient.

37. Shallow vs Deep Copy

Important distinction.

Shallow copy:

```EXAMPLE / FLOW
copies references
```

Deep copy:

```EXAMPLE / FLOW
copies nested object graph
```

Example:

```C#
public sealed record CampaignTemplate(
    List<string> Tags);
```

Using:

```C#
template with { }
```

does not automatically deeply clone `Tags`.

Both records may reference the same list.

38. When Prototype Helps

EXAMPLE / FLOW

```
Expensive base configuration

Many similar objects

Template-based creation

Test data generation

Immutable record variations
```

Avoid it if:

EXAMPLE / FLOW

```
normal constructor is already simple
```

PART 2 — STRUCTURAL PATTERNS

39. Adapter Pattern

Adapter converts one interface into another expected by the client.

Problem:

Your application expects:

C#

```
public interface IPaymentGateway
{
    Task<PaymentResult>
        ChargeAsync(
            Money amount,
            CancellationToken cancellationToken);
}
```

Third-party SDK exposes:

C#

```
VendorPaymentResponse
    MakePayment(
        decimal amount);
```

Interfaces do not match.

40. Adapter Example

C#

```
public sealed class VendorPaymentAdapter :
    IPaymentGateway
{
    private readonly VendorClient _client;

    public VendorPaymentAdapter(
        VendorClient client)
    {
        _client = client;
    }
```

```
    public async Task<PaymentResult>
        ChargeAsync(
            Money amount,
            CancellationToken cancellationToken)
    {
        VendorPaymentResponse response =
            await _client.PayAsync(
                amount.Amount,
                cancellationToken);

        return new PaymentResult(
            response.Success,
            response.TransactionId);
    }
}
```

41. Adapter Flow

EXAMPLE / FLOW

```
Application
    ↓
IPaymentGateway
    ↓
VendorPaymentAdapter
    ↓
Vendor SDK
```

The domain/application never depends directly on the vendor API.

42. Adapter Advantages

EXAMPLE / FLOW

```
Protect application from vendor API

Easy replacement

Easy testing

Transforms models/errors

Centralizes integration quirks
```

43. Adapter vs Facade

Adapter:

EXAMPLE / FLOW

```
changes interface shape
```

Facade:

EXAMPLE / FLOW

```
simplifies access to a subsystem
```

They solve different problems.

44. Bridge Pattern

Bridge separates:

```
EXAMPLE / FLOW
abstraction
```

from:

```
EXAMPLE / FLOW
implementation dimension
```

so both can vary independently.

Suppose receipt format and delivery channel are independent.

Formats:

```
EXAMPLE / FLOW
Standard receipt
Tax receipt
Tribute receipt
```

Delivery:

```
EXAMPLE / FLOW
Email
Print
Portal
```

Without Bridge you might create:

```
EXAMPLE / FLOW
StandardEmailReceipt
StandardPrintReceipt
TaxEmailReceipt
TaxPrintReceipt
TributeEmailReceipt
TributePrintReceipt
```

Class explosion.

45. Bridge Example

```
C#
public interface IReceiptDelivery
{
    Task DeliverAsync(
        ReceiptDocument document,
        CancellationToken cancellationToken);
}
```

Abstraction:

```
C#
public abstract class Receipt
{
    protected readonly
        IReceiptDelivery Delivery;
```

```
    protected Receipt(
        IReceiptDelivery delivery)
    {
        Delivery = delivery;
    }

    public abstract Task SendAsync(
        Gift gift,
        CancellationToken cancellationToken);
}
```

Concrete receipt:

```C#
public sealed class TaxReceipt :
    Receipt
{
    public TaxReceipt(
        IReceiptDelivery delivery)
        :
        base(delivery)
    {
    }

    public override async Task SendAsync(
        Gift gift,
        CancellationToken cancellationToken)
    {
        ReceiptDocument document =
            BuildTaxReceipt(
                gift);

        await Delivery
            .DeliverAsync(
                document,
                cancellationToken);
    }
}
```

Now:

EXAMPLE / FLOW

```
Receipt type
```

and:

EXAMPLE / FLOW

```
delivery mechanism
```

vary independently.

46. Composite Pattern

Composite treats:

EXAMPLE / FLOW

```
single objects
```

and:

through the same abstraction.

Classic tree structure.

47. Fundraising Composite Example

Suppose campaigns can contain:

```
EXAMPLE / FLOW
Campaign
    ├─ Subcampaign
    └─ Subcampaign
        └─ Subcampaign
```

Define:

```
C#
public interface ICampaignNode
{
    decimal CalculateTotal();
}
```

Leaf:

```
C#
public sealed class GiftNode :
    ICampaignNode
{
    private readonly decimal _amount;

    public GiftNode(
        decimal amount)
    {
        _amount = amount;
    }

    public decimal CalculateTotal()
        =>
        _amount;
}
```

Composite:

```
C#
public sealed class CampaignGroup :
    ICampaignNode
{
    private readonly
        List<ICampaignNode>
        _children = [];

    public void Add(
        ICampaignNode node)
    {
        _children.Add(
            node);
    }
```

```
    public decimal CalculateTotal()
    {
        return _children.Sum(
            child =>
                child.CalculateTotal());
    }
}
```

Caller treats both as:

```
C#
ICampaignNode
```

48. Decorator Pattern

Decorator wraps an object implementing the same contract and adds behavior.

Core structure:

```
EXAMPLE / FLOW
Client
   ↓
Decorator
   ↓
Decorated implementation
```

49. Receipt Decorator Example

Interface:

```
C#
public interface IReceiptSender
{
    Task SendAsync(
        Gift gift,
        CancellationToken cancellationToken);
}
```

Core:

```
C#
public sealed class EmailReceiptSender :
    IReceiptSender
{
    public Task SendAsync(
        Gift gift,
        CancellationToken cancellationToken)
    {
        // Send email.

        return Task.CompletedTask;
    }
}
```

Decorator:

```
C#
public sealed class
    AuditingReceiptSender
    :
```

```
    IReceiptSender
{
    private readonly
        IReceiptSender
        _inner;

    private readonly
        IAuditWriter
        _audit;

    public AuditingReceiptSender(
        IReceiptSender inner,
        IAuditWriter audit)
    {
        _inner = inner;

        _audit = audit;
    }

    public async Task SendAsync(
        Gift gift,
        CancellationToken cancellationToken)
    {
        await _inner.SendAsync(
            gift,
            cancellationToken);

        await _audit.WriteAsync(
            gift.GiftId,
            "Receipt sent",
            cancellationToken);
    }
}
```

50. Multiple Decorators

EXAMPLE / FLOW

```
Client
   ↓
MetricsDecorator
   ↓
LoggingDecorator
   ↓
RetryDecorator
   ↓
EmailReceiptSender
```

Each adds one cross-cutting behavior.

51. Decorator vs Inheritance

Inheritance:

EXAMPLE / FLOW

```
behavior fixed in hierarchy
```

Decorator:

EXAMPLE / FLOW

```
behavior composed dynamically
```

Decorator supports:

EXAMPLE / FLOW

```
Logging + Retry

Logging only

Retry + Audit

Audit + Metrics + Retry
```

without creating subclasses for every combination.

52. Decorator in HttpClient

`IHttpClientFactory` supports outgoing middleware using chained `DelegatingHandler` instances. Each handler can perform work before and after the next handler, which gives a decorator/pipeline-like composition model for outgoing HTTP concerns such as logging or resilience.

Example:

C#

```
public sealed class TenantHeaderHandler :
    DelegatingHandler
{
    protected override Task<
        HttpResponseMessage>
        SendAsync(
            HttpRequestMessage request,
            CancellationToken cancellationToken)
    {
        request.Headers.Add(
            "X-Tenant",
            "10");

        return base.SendAsync(
            request,
            cancellationToken);
    }
}
```

Registration:

C#

```
builder.Services
    .AddTransient<
        TenantHeaderHandler>();

builder.Services
    .AddHttpClient<
        PaymentGatewayClient>()
    .AddHttpMessageHandler<
        TenantHeaderHandler>();
```

53. Facade Pattern

Facade provides a:

> **Simple interface over a more complicated subsystem.**

Suppose completing a gift requires:

EXAMPLE / FLOW

```
Validation
Payment
Persistence
Receipt
Audit
Outbox
```

Caller should not necessarily know each subsystem.

54. Facade Example

C#

```
public sealed class GiftProcessingFacade
{
    private readonly
        IGiftValidator
        _validator;

    private readonly
        IPaymentGateway
        _payments;

    private readonly
        IGiftRepository
        _repository;

    private readonly
        IReceiptSender
        _receiptSender;

    public GiftProcessingFacade(
        IGiftValidator validator,
        IPaymentGateway payments,
        IGiftRepository repository,
        IReceiptSender receiptSender)
    {
        _validator = validator;

        _payments = payments;

        _repository = repository;

        _receiptSender = receiptSender;
    }

    public async Task CompleteAsync(
        Gift gift,
        CancellationToken cancellationToken)
    {
        _validator.Validate(
            gift);

        await _payments
            .ChargeAsync(
                gift.TotalAmount,
                cancellationToken);

        gift.Complete();

        await _repository
```

```
            .SaveAsync(
                gift,
                cancellationToken);

        await _receiptSender
            .SendAsync(
                gift,
                cancellationToken);
    }
}
```

Caller sees:

```
C#
facade.CompleteAsync(...)
```

instead of coordinating the entire subsystem.

55. Facade vs Service Layer

They can look similar.

Facade:

```
EXAMPLE / FLOW
Simplifies access to complex subsystem
```

Service Layer:

```
EXAMPLE / FLOW
Defines application's use-case/application boundary
```

A service layer can act like a facade, but conceptually they are not identical.

Topic 35 will cover Service Layer in depth.

56. Flyweight Pattern

Flyweight reduces memory by sharing reusable intrinsic state among many logical objects.

Imagine millions of exported Gift rows all referencing the same:

```
EXAMPLE / FLOW
Currency metadata
Country metadata
Fund category metadata
```

Instead of duplicating immutable data:

```
EXAMPLE / FLOW
Each row → own currency object
```

you share:

```
EXAMPLE / FLOW
USD metadata singleton/value
```

57. Flyweight Concept

Separate:

EXAMPLE / FLOW

```
Intrinsic state
    →
shared
```

from:

EXAMPLE / FLOW

```
Extrinsic state
    →
specific to each use
```

Example:

EXAMPLE / FLOW

```
Shared:
Currency code = USD
Symbol = $

Per gift:
Amount = 100
```

58. Flyweight Example

C#

```
public sealed record CurrencyInfo(
    string Code,
    string Symbol);

public static class CurrencyRegistry
{
    public static readonly
        CurrencyInfo Usd =
            new(
                "USD",
                "$");

    public static readonly
        CurrencyInfo Eur =
            new(
                "EUR",
                "€");
}
```

Each transaction can reuse:

C#

```
CurrencyRegistry.Usd
```

rather than allocating another equivalent metadata object.

59. When Flyweight Is Useful

EXAMPLE / FLOW

```
Huge object counts

Large duplicated immutable metadata

Memory pressure proven by profiling
```

Avoid implementing complex Flyweight infrastructure for:

EXAMPLE / FLOW

```
20 ordinary objects
```

60. Proxy Pattern

Proxy provides a stand-in object controlling access to another object.

Structure:

EXAMPLE / FLOW

```
Client
   ↓
Proxy
   ↓
Real Object
```

Possible reasons:

EXAMPLE / FLOW

```
Authorization
Lazy loading
Remote access
Caching
Logging
Rate limiting
```

61. Proxy Example

C#

```
public interface IGiftDocumentStore
{
    Task<Stream> OpenAsync(
        long giftId,
        CancellationToken cancellationToken);
}
```

Real service:

C#

```
public sealed class GiftDocumentStore :
    IGiftDocumentStore
{
    public Task<Stream> OpenAsync(
        long giftId,
        CancellationToken cancellationToken)
    {
        // Open actual document.
```

```
        throw new
            NotImplementedException();
    }
}
```

Security proxy:

```C#
public sealed class
    AuthorizedGiftDocumentStore
    :
    IGiftDocumentStore
{
    private readonly
        IGiftDocumentStore
        _inner;

    private readonly
        IAuthorizationChecker
        _authorization;

    public AuthorizedGiftDocumentStore(
        IGiftDocumentStore inner,
        IAuthorizationChecker authorization)
    {
        _inner = inner;

        _authorization = authorization;
    }

    public async Task<Stream> OpenAsync(
        long giftId,
        CancellationToken cancellationToken)
    {
        await _authorization
            .EnsureCanReadGiftAsync(
                giftId,
                cancellationToken);

        return await _inner
            .OpenAsync(
                giftId,
                cancellationToken);
    }
}
```

62. Proxy vs Decorator

They can have nearly identical class structure.

Difference is **intent**.

Decorator:

EXAMPLE / FLOW

```
Adds responsibilities/behavior
```

Proxy:

EXAMPLE / FLOW

```
Controls or represents access
to another object
```

DESIGN PATTERNS

Intent determines the pattern.

PART 3 — BEHAVIORAL PATTERNS

63. Strategy Pattern

Strategy encapsulates interchangeable algorithms behind one contract.

This is one of the most useful patterns in C# applications.

64. Bad Strategy Problem

C#

```
decimal CalculateFee(
    PaymentMethod method,
    decimal amount)
{
    switch (method)
    {
        case PaymentMethod.Card:
            return amount * 0.03m;

        case PaymentMethod.Ach:
            return 1m;

        case PaymentMethod.Check:
            return 0m;

        default:
            throw new
                ArgumentOutOfRangeException();
    }
}
```

Every new payment method modifies central logic.

65. Strategy Implementation

C#

```
public interface IPaymentFeeStrategy
{
    PaymentMethod Method
    {
        get;
    }

    decimal Calculate(
        decimal amount);
}
```

Card:

C#

```
public sealed class
    CardFeeStrategy
```

```
    :
    IPaymentFeeStrategy
{
    public PaymentMethod Method =>
        PaymentMethod.Card;

    public decimal Calculate(
        decimal amount)
        =>
        amount * 0.03m;
}
```

ACH:

```C#
public sealed class
    AchFeeStrategy
    :
    IPaymentFeeStrategy
{
    public PaymentMethod Method =>
        PaymentMethod.Ach;

    public decimal Calculate(
        decimal amount)
        =>
        1m;
}
```

66. Strategy Resolver

```C#
public sealed class PaymentFeeCalculator
{
    private readonly
        IReadOnlyDictionary<
            PaymentMethod,
            IPaymentFeeStrategy>
        _strategies;

    public PaymentFeeCalculator(
        IEnumerable<
            IPaymentFeeStrategy>
            strategies)
    {
        _strategies =
            strategies.ToDictionary(
                strategy =>
                    strategy.Method);
    }

    public decimal Calculate(
        PaymentMethod method,
        decimal amount)
    {
        return _strategies[
            method]
            .Calculate(
                amount);
    }
}
```

67. Strategy with Keyed DI

Modern .NET DI supports keyed services, which can be useful where runtime composition needs multiple implementations selected by key.

Example:

```C#
builder.Services
    .AddKeyedScoped<
        IPaymentGateway,
        CardPaymentGateway>(
            PaymentMethod.Card);

builder.Services
    .AddKeyedScoped<
        IPaymentGateway,
        AchPaymentGateway>(
            PaymentMethod.Ach);
```

Use carefully.

Do not scatter container keys throughout domain logic if a typed strategy resolver expresses intent better.

68. Strategy with Delegates

Not every Strategy needs a class hierarchy.

Example:

```C#
Func<decimal, decimal>
    feeCalculator =
        amount =>
            amount * 0.03m;
```

Delegates are an excellent lightweight Strategy mechanism. Microsoft specifically notes that delegates enable designs with minimal coupling and uses LINQ predicates as a major example of this style.

69. Strategy vs Factory

Strategy:

```
EXAMPLE / FLOW
Which algorithm should perform behavior?
```

Factory:

```
EXAMPLE / FLOW
Which object should be created?
```

They frequently work together.

70. Observer Pattern

Observer defines:

```
EXAMPLE / FLOW
Publisher / Subject
    ↓
```

```
notifications
 *  ↓
multiple Subscribers / Observers
```


Example:

EXAMPLE / FLOW

```
Gift Completed
     ↓
 ┌──────────────┬──────────────┐
 ↓              ↓              ↓
Audit          Metrics       Receipt
Observer       Observer      Observer
```

71. Observer in .NET

.NET supports Observer directly with:

C#

```
IObservable<T>
IObserver<T>
```

The provider accepts subscriptions and pushes notifications to observers.

C# events also provide an observer-style notification mechanism.

72. Simple Event Observer Example

C#

```
public sealed class GiftProcessor
{
    public event
        EventHandler<
            GiftCompletedEventArgs>?
        GiftCompleted;

    public void Complete(
        Gift gift)
    {
        gift.Complete();

        GiftCompleted?.Invoke(
            this,
            new GiftCompletedEventArgs(
                gift.GiftId));
    }
}
```

Subscribers:

C#

```
processor.GiftCompleted += auditHandler;

processor.GiftCompleted += metricsHandler;
```

73. Observer Advantages

EXAMPLE / FLOW

```
Publisher does not know every subscriber

Easy extensibility

Good for notification

Supports one-to-many relationships
```

74. Observer Risks

EXAMPLE / FLOW

```
Hidden control flow

Subscription lifetime leaks

Order assumptions

Synchronous handler failures

Difficult debugging

Too many implicit side effects
```

Current .NET Observer guidance explicitly recommends not depending on observer notification order and carefully managing subscriptions/unsubscriptions.

75. Command Pattern

Command encapsulates a request/action as an object.

Instead of:

C#

```
service.CompleteGift(
    giftId);
```

represent intent as:

C#

```
public sealed record
    CompleteGiftCommand(
        long TenantId,
        long GiftId);
```

Handler:

C#

```
public sealed class
    CompleteGiftCommandHandler
{
    public async Task HandleAsync(
        CompleteGiftCommand command,
        CancellationToken cancellationToken)
    {
        // Execute use case.
```

```
    }
}
```


76. Why Command Is Useful

A command object can carry:

EXAMPLE / FLOW

```
Intent

Data

Validation context

Correlation information
```

Commands can be:

EXAMPLE / FLOW

```
Queued
Logged
Retried
Dispatched
Authorized
Tested independently
```

77. Command vs CQRS

Command pattern:

EXAMPLE / FLOW

```
Encapsulates action/request
```

CQRS:

EXAMPLE / FLOW

```
Architectural separation between
commands and queries
```

You can use Command without full CQRS.

Microsoft's CQRS guidance distinguishes commands, which change system state, from queries, which return data without changing state.

Topic 34 will discuss architectural implications in more depth.

78. Chain of Responsibility Pattern

Chain of Responsibility passes a request through a sequence of handlers.

Each handler decides whether to:

EXAMPLE / FLOW

```
Handle

Continue

Reject
```

Modify

79. Validation Chain Example

```C#
public interface IGiftValidationHandler
{
    void SetNext(
        IGiftValidationHandler next);

    void Validate(
        Gift gift);
}
```

Base:

```C#
public abstract class
    GiftValidationHandler
    :
    IGiftValidationHandler
{
    private
        IGiftValidationHandler?
        _next;

    public void SetNext(
        IGiftValidationHandler next)
    {
        _next = next;
    }

    public virtual void Validate(
        Gift gift)
    {
        _next?.Validate(
            gift);
    }
}
```

Amount validator:

```C#
public sealed class
    AmountValidationHandler
    :
    GiftValidationHandler
{
    public override void Validate(
        Gift gift)
    {
        if (
            gift.TotalAmount <= 0)
        {
            throw new
                InvalidOperationException(
                    "Gift amount must be positive.");
        }

        base.Validate(
            gift);
```

```
    }
}
```


80. Chain Flow

EXAMPLE / FLOW

```
Gift
 ↓
Amount Validation
 ↓
Tenant Validation
 ↓
Campaign Validation
 ↓
Allocation Validation
 ↓
Success
```

A handler may terminate the chain by:

EXAMPLE / FLOW

```
throwing

returning result

not invoking next
```

81. ASP.NET Core Middleware and Chain of Responsibility

ASP.NET Core middleware forms a pipeline of request delegates. Each middleware decides whether to call the next component and may perform work before and after it; middleware can also short-circuit processing. This closely resembles Chain of Responsibility, while the before/after wrapping also has decorator-like characteristics.

Example:

C#

```
app.UseAuthentication();

app.UseAuthorization();

app.UseMiddleware<
    TenantValidationMiddleware>();

app.MapControllers();
```

Flow:

EXAMPLE / FLOW

```
Request
  ↓
Authentication
  ↓
Authorization
  ↓
Tenant validation
  ↓
Controller
```

82. Template Method Pattern

Template Method defines the:

EXAMPLE / FLOW

```
overall algorithm skeleton
```

in a base class while allowing subclasses to override specific steps.

83. Template Method Example

C#

```
public abstract class GiftImportProcessor
{
    public async Task ProcessAsync(
        Stream file,
        CancellationToken cancellationToken)
    {
        IReadOnlyList<
            GiftImportRow>
            rows =
                await ParseAsync(
                    file,
                    cancellationToken);

        Validate(
            rows);

        await SaveAsync(
            rows,
            cancellationToken);
    }

    protected abstract Task<
        IReadOnlyList<
            GiftImportRow>>
        ParseAsync(
            Stream file,
            CancellationToken cancellationToken);

    protected virtual void Validate(
        IReadOnlyList<
            GiftImportRow> rows)
    {
        if (
            rows.Count == 0)
        {
            throw new
                InvalidOperationException();
        }
    }

    protected abstract Task SaveAsync(
        IReadOnlyList<
            GiftImportRow> rows,
        CancellationToken cancellationToken);
}
```

CSV implementation:

C#

```
public sealed class CsvGiftImporter :
    GiftImportProcessor
{
    protected override Task<
        IReadOnlyList<
            GiftImportRow>>
        ParseAsync(
            Stream file,
            CancellationToken cancellationToken)
    {
        // CSV parsing.

        throw new
            NotImplementedException();
    }

    protected override Task SaveAsync(
        IReadOnlyList<
            GiftImportRow> rows,
        CancellationToken cancellationToken)
    {
        // Save.

        throw new
            NotImplementedException();
    }
}
```

84. Template Method vs Strategy

Template Method:

EXAMPLE / FLOW

```
Inheritance
```

and base class controls the algorithm.

Strategy:

EXAMPLE / FLOW

```
Composition
```

and caller supplies interchangeable behavior.

Modern design often prefers Strategy when inheritance is not genuinely appropriate.

85. State Pattern

State pattern changes object behavior when its internal state changes.

Instead of:

C#

```
switch (
    Status)
{
    case GiftStatus.Pending:
        ...
        break;

    case GiftStatus.Completed:
```

```
        ...
        break;

    case GiftStatus.Cancelled:
        ...
        break;
}
```

behavior moves into state objects.

86. State Pattern Example

C#

```
public interface IGiftState
{
    void Complete(
        GiftContext gift);

    void Cancel(
        GiftContext gift);
}
```

Pending:

C#

```
public sealed class PendingGiftState :
    IGiftState
{
    public void Complete(
        GiftContext gift)
    {
        gift.ChangeState(
            new CompletedGiftState());
    }

    public void Cancel(
        GiftContext gift)
    {
        gift.ChangeState(
            new CancelledGiftState());
    }
}
```

Completed:

C#

```
public sealed class CompletedGiftState :
    IGiftState
{
    public void Complete(
        GiftContext gift)
    {
        throw new
            InvalidOperationException(
                "Gift already completed.");
    }

    public void Cancel(
        GiftContext gift)
    {
        throw new
            InvalidOperationException(
```

```
                "Completed gift cannot be cancelled.");
    }
}
```


87. State Pattern Flow

EXAMPLE / FLOW

```
Gift Context
    ↓
Pending State
    ↓ Complete()
Completed State
```

Behavior changes because:

EXAMPLE / FLOW

```
current state object changes
```

not because every caller contains another status `switch`.

88. When State Pattern Is Worth It

Use when:

EXAMPLE / FLOW

```
Many states

Many state-specific behaviors

Complex transitions

Large switch statements repeated everywhere
```

Do not implement 12 classes for:

EXAMPLE / FLOW

```
Pending / Completed
```

if one simple method with an enum is clear enough.

89. Mediator Pattern

Mediator centralizes communication between components so they don't depend directly on one another.

Without mediator:

EXAMPLE / FLOW

```
GiftService → AuditService
GiftService → ReceiptService
GiftService → CampaignService
GiftService → MetricsService
```

With mediator:

EXAMPLE / FLOW

```
GiftService
    ↓
Mediator
```

```
 ↓  ↓     ↓      ↓
A  B      C      D
```

90. Simple Mediator Example

```C#
public interface IApplicationMediator
{
    Task PublishAsync<T>(
        T notification,
        CancellationToken cancellationToken);
}
```

Application service:

```C#
await _mediator.PublishAsync(
    new GiftCompletedNotification(
        gift.GiftId),
    cancellationToken);
```

Mediator dispatches to registered handlers.

91. Mediator Advantages

EXAMPLE / FLOW

```
Reduces direct dependencies

Centralizes dispatching

Supports command/notification handlers

Makes extensibility easier
```

92. Mediator Risks

EXAMPLE / FLOW

```
Hidden call graph

Everything depends on mediator

Runtime dispatch errors

Difficult navigation

Can become service locator
```

Do not replace every ordinary method call with a mediator.

If:

```C#
calculator.Calculate()
```

is simple and explicit, keep it.

93. Iterator Pattern

Iterator provides sequential access to elements without exposing collection internals.

C# makes this pattern extremely natural through:

```
C#
IEnumerable<T>
IEnumerator<T>
yield return
foreach
```

94. Iterator Example

```
C#
public IEnumerable<Gift>
    GetMajorGifts(
        IEnumerable<Gift> gifts,
        decimal threshold)
{
    foreach (
        Gift gift
        in gifts)
    {
        if (
            gift.TotalAmount
            >=
            threshold)
        {
            yield return gift;
        }
    }
}
```

Usage:

```
C#
foreach (
    Gift gift
    in GetMajorGifts(
        gifts,
        10_000m))
{
}
```

Caller does not care how iteration is implemented internally.

95. Iterator Benefits

```
EXAMPLE / FLOW
Lazy enumeration

Encapsulation of traversal

Uniform foreach support

No need to expose underlying collection
```

96. Memento Pattern

Memento captures an object's state so it can later be restored without exposing internal representation.

Possible use:

EXAMPLE / FLOW

```
Undo
Rollback
Editor history
Workflow drafts
```

97. Memento Example

C#

```
public sealed record GiftDraftMemento(
    decimal Amount,
    long? CampaignId,
    string? Comments);
```

Gift draft:

C#

```
public sealed class GiftDraft
{
    public decimal Amount
    {
        get;
        private set;
    }

    public long? CampaignId
    {
        get;
        private set;
    }

    public string? Comments
    {
        get;
        private set;
    }

    public GiftDraftMemento
        CreateMemento()
    {
        return new(
            Amount,
            CampaignId,
            Comments);
    }

    public void Restore(
        GiftDraftMemento memento)
    {
        Amount = memento.Amount;

        CampaignId = memento.CampaignId;

        Comments = memento.Comments;
```

```
    }
}
```


98. Memento Caveat

Memento is not equivalent to:

EXAMPLE / FLOW

```
database transaction
```

A transaction provides storage-level atomicity/isolation.

Memento provides:

EXAMPLE / FLOW

```
application object state snapshot
```

99. Visitor Pattern

Visitor lets you add operations over a relatively stable object hierarchy without modifying every concrete type for each new operation.

Example hierarchy:

EXAMPLE / FLOW

```
Gift
Campaign
Fund
```

Visitor:

C#

```
public interface IFundraisingVisitor
{
    void Visit(
        Gift gift);

    void Visit(
        Campaign campaign);

    void Visit(
        Fund fund);
}
```

Each element:

C#

```
public interface IFundraisingElement
{
    void Accept(
        IFundraisingVisitor visitor);
}
```

Gift:

C#

```
public void Accept(
    IFundraisingVisitor visitor)
{
```

```
    visitor.Visit(
        this);
}
```


100. Visitor Strength

Visitor is useful when:

EXAMPLE / FLOW

```
Object hierarchy changes rarely

New operations are added frequently
```

Examples:

EXAMPLE / FLOW

```
Exporting
Reporting
Auditing
Serialization
Diagnostics
```

101. Visitor Weakness

Adding a new element type requires changing:

EXAMPLE / FLOW

```
visitor interface

every visitor implementation
```

Therefore Visitor favors:

EXAMPLE / FLOW

```
stable element hierarchy
```

but:

EXAMPLE / FLOW

```
changing operations
```

102. Interpreter Pattern

Interpreter represents a grammar and evaluates expressions in that grammar.

Example requirement:

EXAMPLE / FLOW

```
Gift amount > 1000
AND
Campaign = "Annual"
```

You could create an expression language:

EXAMPLE / FLOW

```
amount > 1000 AND campaign = annual
```

and parse it into expression objects.

103. Simplified Interpreter

```C#
public interface IGiftExpression
{
    bool Interpret(
        Gift gift);
}
```

Amount expression:

```C#
public sealed class
    MinimumAmountExpression
    :
    IGiftExpression
{
    private readonly decimal _minimum;

    public MinimumAmountExpression(
        decimal minimum)
    {
        _minimum = minimum;
    }

    public bool Interpret(
        Gift gift)
        =>
        gift.TotalAmount
        >= _minimum;
}
```

AND:

```C#
public sealed class
    AndExpression
    :
    IGiftExpression
{
    private readonly IGiftExpression _left;
    private readonly IGiftExpression _right;

    public AndExpression(
        IGiftExpression left,
        IGiftExpression right)
    {
        _left = left;

        _right = right;
    }

    public bool Interpret(
        Gift gift)
        =>
        _left.Interpret(gift)
        &&
        _right.Interpret(gift);
}
```

104. Interpreter Warning

Do not build your own programming language just because:

EXAMPLE / FLOW

```
Interpreter pattern exists.
```

For many rules:

EXAMPLE / FLOW

```
delegates
expression trees
specifications
configuration
```

may be simpler.

Interpreter is useful when a real domain-specific grammar exists.

PART 4 — PATTERNS FREQUENTLY SEEN IN .NET BACKENDS

105. Repository Pattern

Repository abstracts persistence behind collection/domain-oriented operations.

Example:

C#

```
public interface IGiftRepository
{
    Task<Gift?> FindAsync(
        long giftId,
        CancellationToken cancellationToken);

    Task AddAsync(
        Gift gift,
        CancellationToken cancellationToken);
}
```

Infrastructure:

C#

```
public sealed class GiftRepository :
    IGiftRepository
{
    private readonly
        FundraisingDbContext
        _dbContext;

    // EF implementation.
}
```

Microsoft's architecture guidance describes Repository as an intermediary between domain logic and data mapping/persistence concerns, while also noting that EF Core's `DbContext` already embodies Repository and Unit-of-Work concepts.

Topic 35 will cover Repository in depth.

106. Unit of Work Pattern

Unit of Work groups related changes into one logical persistence operation.

Conceptually:

EXAMPLE / FLOW

```
Change Gift
Change Allocations
Create OutboxMessage
       ↓
One transaction / SaveChanges boundary
```

EF Core `DbContext` already acts as a Unit of Work for many applications.

Topic 35 will explore when wrapping it is useful and when it is redundant.

107. Service Layer Pattern

Service Layer defines application use cases.

Example:

EXAMPLE / FLOW

```
CreateGiftService

CompleteGiftService

RefundGiftService
```

It orchestrates:

EXAMPLE / FLOW

```
Domain
Repositories
External integrations
Transactions
```

without putting all behavior in:

EXAMPLE / FLOW

```
Controllers
```

Topic 35 will cover it deeply.

108. Specification Pattern

Specification encapsulates a reusable business predicate/query rule.

Example:

EXAMPLE / FLOW

```
"Completed gifts over $10,000
for active campaigns"
```

could become:

C#

```
public sealed class MajorCompletedGiftSpecification
{
```

```
    public bool IsSatisfiedBy(
        Gift gift)
    {
        return
            gift.Status
            ==
            GiftStatus.Completed
            &&
            gift.TotalAmount
            >= 10_000m;
    }
}
```

Specification can be useful when business predicates need:

EXAMPLE / FLOW

```
Reuse
Composition
Naming
Persistence translation
```

but don't create specification objects for trivial one-off conditions.

109. Options Pattern

The .NET Options pattern provides strongly typed access to related configuration settings. Microsoft explicitly connects Options to encapsulation/ISP and separation of concerns.

Example:

C#

```
public sealed class ReceiptOptions
{
    public const string SectionName = "Receipt";

    public decimal
        MajorGiftThreshold
    {
        get;
        init;
    }
}
```

Registration:

C#

```
builder.Services
    .Configure<
        ReceiptOptions>(
            builder.Configuration
                .GetSection(
                    ReceiptOptions
                        .SectionName));
```

110. Middleware Pipeline Pattern

ASP.NET Core's request pipeline consists of request delegates chained together. Each component may execute before/after the next or terminate the pipeline.

That pipeline combines ideas closely related to:

EXAMPLE / FLOW

```
Chain of Responsibility

Decorator-style wrapping
```

Example:

EXAMPLE / FLOW

```
Exception Handler
    ↓
Authentication
    ↓
Authorization
    ↓
Rate Limiting
    ↓
Routing / Endpoint
```

111. CQRS Pattern

CQRS separates:

EXAMPLE / FLOW

```
Commands
    →
write/change state
```

from:

EXAMPLE / FLOW

```
Queries
    →
return data
```

Microsoft describes the simplified form as grouping query and command operations separately, without requiring separate physical databases or Event Sourcing.

Example:

EXAMPLE / FLOW

```
CreateGiftCommand
CompleteGiftCommand

GetGiftByIdQuery
SearchGiftsQuery
```

CQRS is an architectural/application pattern and should not automatically be applied to every CRUD system.

112. Outbox Pattern — Boundary Note

A production workflow might need:

EXAMPLE / FLOW

```
Save Gift
+
Publish GiftCompleted integration event
```

The dangerous sequence:

EXAMPLE / FLOW

```
Save database
    ↓
Crash
    ↓
Never publish message
```

can be addressed with a transactional Outbox approach:

EXAMPLE / FLOW

```
Save Gift
+
Save OutboxMessage
    ↓
commit
    ↓
background publisher
```

Your Fundraising design already has a natural future:

EXAMPLE / FLOW

```
integration.OutboxMessage
```

boundary.

We will discuss this more under production architecture in Topic 40 rather than turning this topic into distributed messaging.

PART 5 — PATTERN COMPARISONS

113. Factory vs Builder

Factory:

EXAMPLE / FLOW

```
Which object should I create?
```

Builder:

EXAMPLE / FLOW

```
How do I construct a complex object step-by-step?
```

114. Factory vs Strategy

Factory:

EXAMPLE / FLOW

```
Object creation variation
```

Strategy:

EXAMPLE / FLOW

```
Algorithm/behavior variation
```

115. Adapter vs Decorator

Adapter:

EXAMPLE / FLOW

```
Change interface
```

Decorator:

EXAMPLE / FLOW

```
Preserve interface
+
add behavior
```

116. Adapter vs Facade

Adapter:

EXAMPLE / FLOW

```
Makes incompatible interface compatible
```

Facade:

EXAMPLE / FLOW

```
Provides simpler API over complex subsystem
```

117. Decorator vs Proxy

Both may wrap the same interface.

Decorator intent:

EXAMPLE / FLOW

```
Add behavior
```

Proxy intent:

EXAMPLE / FLOW

```
Control access
```

118. Strategy vs State

Both may use polymorphic objects.

Strategy:

EXAMPLE / FLOW

```
Caller/context selects algorithm
```

State:

EXAMPLE / FLOW

```
Object behavior changes because
internal state changes
```

119. Strategy vs Template Method

Strategy:

EXAMPLE / FLOW

```
Composition
```

Template Method:

EXAMPLE / FLOW

```
Inheritance
```

Strategy allows behavior replacement without subclassing.

120. Observer vs Mediator

Observer:

EXAMPLE / FLOW

```
Publisher pushes notifications
to subscribers
```

Mediator:

EXAMPLE / FLOW

```
Communication between components
is routed through central coordinator
```

121. Chain of Responsibility vs Decorator

Chain:

EXAMPLE / FLOW

```
Each handler may decide whether
processing continues
```

Decorator:

EXAMPLE / FLOW

```
Normally wraps same operation
and delegates to inner object
```

ASP.NET Core middleware exhibits characteristics of both.

122. Command vs Strategy

Command:

EXAMPLE / FLOW

```
Encapsulates "what should happen"
```

Strategy:

EXAMPLE / FLOW

```
Encapsulates "how behavior is performed"
```

Example:

EXAMPLE / FLOW

```
CompleteGiftCommand
    →
intent

PaymentFeeStrategy
    →
algorithm
```

123. Pattern Selection Table

Problem	Pattern candidate
Need one shared object	Singleton
Hide object creation	Factory
Create related object family	Abstract Factory
Complex staged construction	Builder
Clone configured object	Prototype
Adapt third-party API	Adapter
Two dimensions vary independently	Bridge
Tree of individual/group objects	Composite
Add cross-cutting behavior	Decorator
Simplify complex subsystem	Facade
Share repeated immutable state	Flyweight
Control object access	Proxy
Pipeline of handlers	Chain of Responsibility
Encapsulate request	Command
Domain-specific grammar	Interpreter
Traverse collection	Iterator
Centralize communication	Mediator
Save/restore state	Memento
One-to-many notifications	Observer
Behavior varies with state	State
Swap algorithms	Strategy
Algorithm skeleton with overridable steps	Template Method
Add operations to stable hierarchy	Visitor

PART 6 — ASP.NET CORE PATTERN MAPPING

124. Common .NET Pattern Relationships

A modern .NET application frequently contains pattern-like structures such as:

EXAMPLE / FLOW

```
Dependency Injection
    →
composition / IoC

IHttpClientFactory
    →
Factory-oriented API

DelegatingHandler
    →
Decorator / pipeline behavior

Middleware
    →
Chain of Responsibility + wrapping

Events / IObservable<T>
    →
Observer

IEnumerable<T> / yield
    →
Iterator

Options
    →
Options pattern

Repositories
    →
Repository

DbContext
    →
Repository + Unit-of-Work concepts

Multiple DI implementations
    →
Strategy-friendly composition
```

Remember:

> ⓘ Similar structure does not mean every framework class is a textbook GoF implementation.

Intent matters.

125. Patterns and DI Work Together

Example:

C#

```
builder.Services
    .AddScoped<
```

```
        IPaymentFeeStrategy,
        CardFeeStrategy>();

builder.Services
    .AddScoped<
        IPaymentFeeStrategy,
        AchFeeStrategy>();

builder.Services
    .AddScoped<
        PaymentFeeCalculator>();
```

DI handles:

EXAMPLE / FLOW

```
construction
```

Strategy handles:

EXAMPLE / FLOW

```
behavior variation
```

These solve different problems.

PART 7 — FUNDRAISING PLATFORM END-TO-END EXAMPLE

126. Requirement

Suppose Gift processing requires:

EXAMPLE / FLOW

```
Validate Gift

Select payment provider

Convert provider API

Charge payment

Complete Gift

Persist Gift

Write OutboxMessage

Send receipt

Audit receipt delivery
```

Different patterns naturally solve different concerns.

127. Validation — Chain of Responsibility

EXAMPLE / FLOW

```
Gift
 ↓
Tenant Validator
 ↓
Amount Validator
```

```
 ↓
Campaign Validator
 ↓
Allocation Validator
```

Each validator handles one part.

128. Payment Selection — Strategy

```C#
public interface IPaymentStrategy
{
    PaymentMethod Method
    {
        get;
    }

    Task<PaymentResult>
        ChargeAsync(
            Gift gift,
            CancellationToken cancellationToken);
}
```

Implementations:

```
CardPaymentStrategy

AchPaymentStrategy

CheckPaymentStrategy
```

129. Vendor Integration — Adapter

```
Application
    ↓
IPaymentGateway
    ↓
VendorPaymentAdapter
    ↓
Vendor SDK/API
```

The application remains independent of vendor-specific request/response models.

130. Receipt — Decorator

Base:

```
EmailReceiptSender
```

Wrapped:

```
MetricsReceiptDecorator
        ↓
AuditReceiptDecorator
```

```
        ↓
EmailReceiptSender
```

Cross-cutting behavior is composed rather than placed inside the email sender.

131. Gift State — State Pattern Only If Needed

Simple version:

```C#
GiftStatus Status
```

plus methods:

```C#
gift.Complete();

gift.Cancel();
```

is probably enough.

If transitions become highly complicated:

EXAMPLE / FLOW

```
Draft
Pending
Processing
Completed
Reversed
Cancelled
Failed
Suspended
```

with state-specific behavior everywhere, State pattern may become worthwhile.

This is an important lesson:

> **Do not apply a pattern until complexity justifies it.**

132. Completion Notification — Observer

EXAMPLE / FLOW

```
GiftCompleted
    ↓
Audit Observer
Metrics Observer
Application Notification Observer
```

For durable cross-process workflows, do not rely only on in-memory Observer/events.

Use durable messaging/outbox patterns where required.

133. Application Boundary — Facade / Service Layer

```C#
public sealed class CompleteGiftService
{
```

```
    public Task CompleteAsync(...)
}
```


provides one application-level entry point for the whole use case.

134. Persistence — Repository / Unit of Work

EXAMPLE / FLOW

```
CompleteGiftService
        ↓
IGiftRepository
        ↓
GiftRepository
        ↓
FundraisingDbContext
        ↓
SQL Server
```

`DbContext` naturally provides unit-of-work style change tracking and commit behavior, while a domain repository may provide a domain-specific persistence boundary where useful.

135. Combined Flow

EXAMPLE / FLOW

```
POST /gifts/{id}/complete
          ↓
CompleteGiftService
          ↓
Validation Chain
          ↓
Payment Strategy
          ↓
Payment Adapter
          ↓
Gift.Complete()
          ↓
Repository / DbContext
          ↓
Save Gift + OutboxMessage
          ↓
Commit
          ↓
Receipt Decorator Pipeline
          ↓
Audit / Metrics / Send
```

Notice:

EXAMPLE / FLOW

```
No pattern owns the entire application.
```

Each pattern solves one specific design problem.

PART 8 — PERFORMANCE CONSIDERATIONS

136. Patterns Have Costs

Patterns may introduce:

EXAMPLE / FLOW

```
More classes

More interfaces

More indirection

More allocations

More virtual/interface calls

More object graph complexity
```

Usually:

EXAMPLE / FLOW

```
database
network
serialization
```

costs dominate ordinary pattern indirection in Web APIs.

But do not ignore architecture overhead in extremely hot code.

137. Strategy Performance

A dictionary strategy lookup:

C#

```
_strategyMap[
    type]
```

is usually negligible.

But:

EXAMPLE / FLOW

```
reflection scan every request
```

to find strategies may not be.

Prefer building lookup structures once when useful.

138. Decorator Depth

Ten nested decorators mean:

EXAMPLE / FLOW

```
10 method calls
```

before reaching the core implementation.

Usually fine for I/O operations.

But excessive decorators can make debugging harder.

139. Flyweight and Pooling

Do not confuse:

EXAMPLE / FLOW
```
Flyweight
```

with:

EXAMPLE / FLOW
```
Object Pool
```

Flyweight:

EXAMPLE / FLOW
```
share immutable/reusable intrinsic state
```

Pool:

EXAMPLE / FLOW
```
reuse expensive mutable instances
```

Different intent.

PART 9 — MEMORY CONSIDERATIONS

140. Singleton Lifetime

Singleton may keep everything it references alive:

EXAMPLE / FLOW
```
Singleton
    ↓
Dictionary
    ↓
Gift graph
```

This connects directly to Topic 30.

A badly designed singleton cache can create a managed memory leak.

141. Observer Lifetime

Publisher:

EXAMPLE / FLOW
```
long-lived
```

Subscriber:

EXAMPLE / FLOW
```
short-lived
```

can create:

EXAMPLE / FLOW
```
Publisher
  ↓
delegate
  ↓
subscriber
```

and unintentionally keep subscriber alive.

142. Decorator Object Graph

Decorator chains create an object graph:

EXAMPLE / FLOW

```
A → B → C → Core
```

If registered singleton:

EXAMPLE / FLOW

```
entire graph becomes long-lived
```

Choose service lifetimes intentionally.

PART 10 — THREAD-SAFETY CONSIDERATIONS

143. Pattern Does Not Guarantee Thread Safety

Strategy:

EXAMPLE / FLOW

```
not automatically thread-safe
```

Singleton:

EXAMPLE / FLOW

```
not automatically thread-safe
```

Observer:

EXAMPLE / FLOW

```
not automatically thread-safe
```

Decorator:

EXAMPLE / FLOW

```
not automatically thread-safe
```

Proxy:

EXAMPLE / FLOW

```
not automatically thread-safe
```

Patterns describe object collaboration.

Concurrency rules still apply.

144. Singleton Special Rule

Because all requests may share a singleton:

EXAMPLE / FLOW

```
Singleton + mutable state
    →
requires synchronization
```

or preferably:

EXAMPLE / FLOW

```
immutable/shared-read design
```

145. Observer Concurrency

If observers may subscribe/unsubscribe while another thread publishes:

EXAMPLE / FLOW

```
subscription data structure
```

must be safely coordinated.

Current .NET Observer guidance explicitly recommends thread-safe subscription and disposal behavior for concurrent implementations.

PART 11 — SECURITY CONSIDERATIONS

146. Adapter Security Boundary

Adapter is a good place to normalize:

EXAMPLE / FLOW

```
external errors
external models
external status codes
```

but it should not automatically trust third-party responses.

Validate data crossing the integration boundary.

147. Proxy and Authorization

Authorization proxy can centralize:

EXAMPLE / FLOW

```
access checks
```

but authorization must remain impossible to bypass.

If callers can access:

EXAMPLE / FLOW

```
real implementation
```

directly:

EXAMPLE / FLOW

```
proxy security
```

is ineffective.

148. Strategy and Tenant Isolation

Avoid selecting strategies directly from unvalidated client input:

```
C#
provider.GetKeyedService(
    request.StrategyName);
```

unless allowed keys are validated.

External input should not control arbitrary service resolution.

149. Singleton Security Risk

Never store:

```
EXAMPLE / FLOW
current TenantId
current UserId
authorization decision
request token
```

in normal mutable singleton state.

150. Decorator Security

Decorator is useful for:

```
EXAMPLE / FLOW
authorization
audit
validation
```

but do not assume wrapping only one pathway secures every possible call path.

Security should be enforced at appropriate trust boundaries.

PART 12 — COMMON PATTERN MISTAKES

151. Pattern Fever

Bad:

```
EXAMPLE / FLOW
Create 15 patterns
for 3 classes.
```

Simple code should remain simple.

152. Factory for Every Constructor

Bad:

```
EXAMPLE / FLOW
GiftServiceFactory
```

when DI already constructs GiftService normally.

153. Singleton for Convenience

Bad reason:

EXAMPLE / FLOW

```
"I need access everywhere."
```

That creates global state.

Use explicit dependency injection.

154. Strategy for One Algorithm

If there is exactly one behavior and no meaningful variation:

EXAMPLE / FLOW

```
simple class/method
```

is often better.

155. Decorator for Core Business Logic

Decorators work best for:

EXAMPLE / FLOW

```
cross-cutting behavior
```

Do not bury essential business sequencing inside 12 decorators unless that structure genuinely models the problem.

156. Observer for Mandatory Workflow

Bad:

EXAMPLE / FLOW

```
GiftCompleted event
    →
some unknown subscriber
must save payment transaction
```

If success of the operation requires payment storage:

EXAMPLE / FLOW

```
explicit orchestration
```

is clearer.

Observer is better for:

EXAMPLE / FLOW

```
notification
```

than mandatory invariant-preserving logic.

157. State Pattern Too Early

Three simple enum states do not automatically require three classes.

Start simple.

Refactor when branching becomes difficult.

158. Abstract Factory Too Early

If you have:

EXAMPLE / FLOW

```
one product family
```

an abstract factory may add needless complexity.

159. Mediator Everywhere

Bad:

EXAMPLE / FLOW

```
Controller
  ↓
Mediator
  ↓
Handler
  ↓
Mediator
  ↓
Another handler
  ↓
Mediator
```

until no one can trace execution.

Use mediator where it meaningfully reduces coupling.

160. Repository over EF for No Benefit

Bad generic repository:

C#

```
GetAll()
Add()
Update()
Delete()
```

that simply mirrors:

C#

```
DbSet<T>
```

without domain meaning may add little value.

Microsoft's architecture guidance explicitly notes that EF `DbContext` already implements repository/unit-of-work concepts; a custom repository is most useful when it provides a meaningful domain persistence abstraction.

161. Inheritance Instead of Strategy

Bad:

EXAMPLE / FLOW

```
CardGiftService
AchGiftService
CheckGiftService
```

only to vary fee calculation.

Composition with Strategy is usually clearer.

162. Patterns Without Tests

Patterns introduce abstraction boundaries.

Test behavior through the contract.

Example:

EXAMPLE / FLOW

```
IPaymentFeeStrategy
```

should have behavior tests showing:

EXAMPLE / FLOW

```
correct fee
edge cases
invalid values
```

not merely:

EXAMPLE / FLOW

```
class can be constructed.
```

PART 13 — BEST PRACTICES

163. Design From the Problem

Start with:

EXAMPLE / FLOW

```
What changes?

What varies?

What must remain stable?

What responsibilities are coupled?
```

Then consider a pattern.

164. Prefer Composition

Patterns such as:

EXAMPLE / FLOW

```
Strategy
Decorator
Adapter
Bridge
```

favor composition.

This often gives more flexibility than large inheritance trees.

165. Keep Interfaces Small and Meaningful

Do not create:

EXAMPLE / FLOW

```
IThing
```

for every class.

Create interfaces around:

EXAMPLE / FLOW

```
behavior
capabilities
architecture boundaries
replaceable implementations
```

166. Use Framework Features

Do not rebuild:

EXAMPLE / FLOW

```
Iterator
Observer
DI container
HTTP handler pipeline
middleware pipeline
```

from scratch when .NET already provides appropriate mechanisms.

167. Use Pattern Names for Communication

Good pull-request discussion:

EXAMPLE / FLOW

```
"This class is adapting the vendor API."

"This validation pipeline is Chain of Responsibility."

"This sender is a Decorator."

"This fee calculator uses Strategy."
```

Pattern terminology helps teams communicate.

168. But Document Intent

Do not merely write:

EXAMPLE / FLOW

```
Factory
```

Explain:

EXAMPLE / FLOW

```
Why this factory exists

What variation it owns

What it does not own
```

PART 14 — PRACTICAL EXERCISES

Exercise 1 — Strategy

Refactor:

C#

```
decimal CalculateFee(
    PaymentMethod method,
    decimal amount)
{
    return method switch
    {
        PaymentMethod.Card
            =>
            amount * .03m,

        PaymentMethod.Ach
            =>
            1m,


        _
            =>
            0m
    };
}
```

into:

EXAMPLE / FLOW

```
IPaymentFeeStrategy
CardFeeStrategy
AchFeeStrategy
```

Then register them using DI.

Exercise 2 — Factory

Create:

C#

```
IReceiptSenderFactory
```

that returns:

EXAMPLE / FLOW

```
EmailReceiptSender
PrintReceiptSender
```

based on:

```
C#
ReceiptType
```

Do not inject `IServiceProvider` into the application service.

Exercise 3 — Adapter

Given vendor API:

```
C#
VendorResult
    Process(
        decimal amount);
```

adapt it into:

```
C#
IPaymentGateway.ChargeAsync(...)
```

and translate vendor exceptions to application-specific integration exceptions.

Exercise 4 — Decorator

Start with:

```
C#
IReceiptSender
```

Add:

```
EXAMPLE / FLOW
AuditReceiptSenderDecorator
```

that records:

```
EXAMPLE / FLOW
GiftId
Timestamp
Outcome
```

while preserving the same interface.

Exercise 5 — Chain of Responsibility

Create validators:

```
EXAMPLE / FLOW
TenantValidator
GiftAmountValidator
CampaignValidator
AllocationValidator
```

Then chain them.

Question:

Should the chain stop after the first failure or aggregate errors?

Answer:

EXAMPLE / FLOW

That is part of the chain's contract/design.

Exercise 6 — Observer

Create:

```C#
GiftCompleted
```

event.

Subscribe:

EXAMPLE / FLOW

```
Audit handler
Metrics handler
```

Then unsubscribe one handler and verify it no longer receives events.

Exercise 7 — Builder

Create a:

```C#
GiftDraftBuilder
```

for test data:

```C#
Gift gift =
    new GiftDraftBuilder()
        .ForTenant(1)
        .ForConstituent(10)
        .WithAmount(500m)
        .Build();
```

Keep business invariants inside `Gift`, not only Builder.

Exercise 8 — State

Start with:

```C#
GiftStatus
```

and repeated switch statements.

Refactor into state objects only after state behavior becomes large enough to justify them.

Compare readability before and after.

Exercise 9 — Facade

Build:

C#
```
GiftProcessingFacade
```

that coordinates:

EXAMPLE / FLOW
```
validator
payment
repository
receipt
```

Controller should call only:

C#
```
facade.CompleteAsync(...)
```

for the use case.

Exercise 10 — Proxy

Create:

C#
```
AuthorizedGiftReader
```

wrapping:

C#
```
IGiftReader
```

and require tenant authorization before forwarding the request.

Exercise 11 — Composite

Build hierarchy:

EXAMPLE / FLOW
```
Portfolio
 ├─ Campaign
 │   ├─ Gift
 │   └─ Gift
 └─ Campaign
     └─ Gift
```

Expose:

C#
```
decimal CalculateTotal()
```

through one common abstraction.

Exercise 12 — Pattern Selection

For each requirement choose the likely pattern:

EXAMPLE / FLOW

```
Need different tax algorithms
    →
Strategy

Need wrap email sender with logging
    →
Decorator

Need third-party API compatibility
    →
Adapter

Need one simple entry point over many subsystems
    →
Facade

Need request pipeline
    →
Chain of Responsibility

Need one-to-many notifications
    →
Observer

Need complex staged object construction
    →
Builder
```

PART 15 — EXACTLY 25 INTERVIEW QUESTIONS

1. What is a design pattern?

A design pattern is a reusable design approach to a recurring software-design problem.

It describes:

EXAMPLE / FLOW

```
context
problem
structure
collaborators
trade-offs
```

rather than providing a finished copy-and-paste implementation.

The same pattern may have very different C# implementations depending on the application.

2. What are the three categories of GoF patterns?

The three categories are:

EXAMPLE / FLOW

```
Creational
Structural
Behavioral
```

Creational patterns address object creation.

Structural patterns address composition and relationships among objects.

Behavioral patterns address communication and changing behavior.

3. What is the Singleton pattern and what is its main risk?

Singleton ensures one logical shared instance within a lifetime boundary.

In ASP.NET Core, DI:

```
C#
AddSingleton<T>()
```

normally provides the cleaner implementation style.

The main risk is shared long-lived mutable state.

Because singleton services may be called concurrently, mutable singleton state requires thread-safety and can accidentally retain large object graphs or request-specific information.

4. What is Factory Method?

Factory Method encapsulates the creation of an object so clients don't need to know the concrete class or construction details.

For example:

```
C#
IReceiptSender Create(
    ReceiptType type)
```

may return an Email or Print sender.

Use Factory when creation itself represents meaningful runtime variation.

5. What is the difference between Factory Method and Abstract Factory?

Factory Method usually creates one kind of product.

Abstract Factory creates a related family of products.

For example:

```
EXAMPLE / FLOW
StripeFactory
    →
StripePaymentGateway
StripeRefundGateway

AdyenFactory
    →
AdyenPaymentGateway
AdyenRefundGateway
```

The family remains internally consistent.

6. What is the Builder pattern?

Builder separates complex staged construction from the final object.

It is useful when:

```
EXAMPLE / FLOW
many optional values
staged configuration
readable fluent construction
complex validation/construction
```

exist.

It is unnecessary for simple DTO initialization.

7. What is the Prototype pattern?

Prototype creates an object by copying an existing configured instance.

C# records provide a convenient prototype-like mechanism using:

```
C#
with
```

expressions.

However, developers must understand shallow vs deep copying because nested mutable references may still be shared.

8. What is the Adapter pattern?

Adapter converts an external/incompatible interface into the interface expected by your application.

Example:

```
EXAMPLE / FLOW
IPaymentGateway
    ↓
VendorPaymentAdapter
    ↓
Third-party SDK
```

It is especially useful at integration boundaries because it prevents vendor models and APIs from leaking throughout business logic.

9. What is the Decorator pattern?

Decorator wraps an object implementing the same abstraction and adds behavior before or after delegating to the wrapped object.

For example:

```
EXAMPLE / FLOW
AuditDecorator
    ↓
LoggingDecorator
    ↓
EmailReceiptSender
```

It is useful for cross-cutting concerns while preserving the original contract.

10. What is the difference between Decorator and Proxy?

Their structure can look similar.

Decorator's intent is:

```
EXAMPLE / FLOW
add responsibilities/behavior
```

Proxy's intent is:

EXAMPLE / FLOW

```
control or represent access
```

Examples:

EXAMPLE / FLOW

```
Logging decorator

Authorization proxy
```

Intent determines the pattern.

11. What is the Facade pattern?

Facade provides a simpler entry point over a complex subsystem.

For example:

C#

```
giftProcessingFacade
    .CompleteAsync(...)
```

might internally coordinate:

EXAMPLE / FLOW

```
validation
payment
repository
receipt
audit
```

without requiring callers to know every subsystem.

12. What is the Composite pattern?

Composite allows individual objects and groups of objects to be treated through the same abstraction.

It is useful for tree structures such as:

EXAMPLE / FLOW

```
Campaign portfolio
    ↓
Campaign group
    ↓
Individual Gifts
```

where every node supports an operation such as:

C#

```
CalculateTotal()
```

13. What is the Strategy pattern?

Strategy encapsulates interchangeable algorithms behind one abstraction.

Example:

EXAMPLE / FLOW

```
IPaymentFeeStrategy
    ├─ CardFeeStrategy
```

```
    ├─ AchFeeStrategy
    └─ CheckFeeStrategy
```

The calling component selects the appropriate strategy instead of containing one large conditional block.

Delegates can also serve as lightweight strategies in C#.

14. What is the difference between Strategy and State?

Strategy represents interchangeable algorithms.

The context or caller generally chooses the strategy.

State represents behavior that changes as the object transitions between internal states.

Example:

EXAMPLE / FLOW

```
Strategy:
How should payment fee be calculated?

State:
What operations are allowed while Gift is Pending?
```

15. What is the Observer pattern?

Observer provides one-to-many notification.

A subject/provider publishes changes and zero or more observers receive notifications.

.NET provides:

C#

```
IObservable<T>
IObserver<T>
```

and C# events are also commonly used for observer-style notifications.

16. What is the Command pattern?

Command encapsulates a request or action as an object.

Example:

C#

```
CompleteGiftCommand
```

can carry:

EXAMPLE / FLOW

```
GiftId
TenantId
Correlation information
```

A handler then performs the operation.

Commands can be logged, queued, authorized, validated, or dispatched independently.

17. What is Chain of Responsibility?

Chain of Responsibility passes a request through a series of handlers.

Each handler can:

EXAMPLE / FLOW

```
process
continue
modify
reject
or stop
```

ASP.NET Core middleware strongly resembles this model because each middleware can invoke the next component or short-circuit the pipeline.

18. What is the Template Method pattern?

Template Method defines an algorithm skeleton in a base class while allowing subclasses to override selected steps.

Example:

EXAMPLE / FLOW

```
Parse
Validate
Save
```

can be fixed by the base import processor while CSV and Excel importers implement the parsing steps.

Its primary mechanism is inheritance.

19. What is the difference between Template Method and Strategy?

Template Method uses inheritance:

EXAMPLE / FLOW

```
base algorithm
+
overridden steps
```

Strategy uses composition:

EXAMPLE / FLOW

```
context
+
interchangeable strategy object
```

Strategy is often more flexible because behavior can be swapped without changing inheritance hierarchy.

20. What is the Mediator pattern?

Mediator centralizes communication between components so they don't directly depend on every other collaborator.

Instead of:

EXAMPLE / FLOW

```
A → B
A → C
A → D
```

use:

EXAMPLE / FLOW

```
A → Mediator → handlers
```

This can reduce coupling, but excessive mediator usage can hide control flow and create runtime indirection.

21. What is the Iterator pattern and how does C# support it?

Iterator provides a uniform way to traverse a sequence without exposing the collection's implementation.

C# supports this directly through:

```
C#
IEnumerable<T>
IEnumerator<T>
foreach
yield return
```

For example:

```
C#
IEnumerable<Gift>
    GetMajorGifts(...)
```

can lazily yield matching gifts without exposing storage details.

22. When should you use State instead of a simple enum?

Use an enum and straightforward methods while state logic remains simple.

Consider State pattern when:

```
EXAMPLE / FLOW
many states exist

each state has significant behavior

transition rules are complicated

the same switch statements appear repeatedly
```

Do not create state classes merely because an enum exists.

23. How are Repository and Unit of Work related to EF Core?

Repository abstracts persistence access to domain/application code.

Unit of Work coordinates changes and commits them as one logical persistence operation.

Microsoft's architecture guidance explicitly notes that EF Core's `DbContext` already implements Repository and Unit-of-Work concepts. Therefore, custom repository/UoW wrappers should add meaningful domain or architectural value rather than simply duplicating EF Core APIs.

24. How do you decide whether to use a design pattern?

Start with the problem, not the pattern.

Ask:

```
EXAMPLE / FLOW
What varies?

What changes frequently?

What needs to remain stable?
```

```
Where is coupling hurting us?

Is there duplicated branching?

Is object creation complicated?

Is communication too tightly connected?
```


Then select the simplest pattern that addresses the problem.

If normal methods and composition are already clear, don't introduce a pattern merely for formality.

25. Which patterns would you commonly use in the Fundraising Platform?

A practical mapping could be:

EXAMPLE / FLOW

```
Strategy
    →
payment fee or receipt behavior

Factory
    →
runtime creation/selection when meaningful

Adapter
    →
third-party payment/CRM APIs

Decorator
    →
audit/logging/metrics around integrations

Facade / Service Layer
    →
Gift use-case orchestration

Chain of Responsibility
    →
validation or ASP.NET Core middleware

Observer
    →
in-process notifications

Repository
    →
domain-oriented persistence abstraction

Unit of Work
    →
DbContext transaction/change boundary

Builder
    →
complex Gift drafts/test objects

Proxy
    →
authorization-controlled access

State
    →
complex Gift lifecycle only if needed
```

```
Command
    →
application requests/CQRS-style commands
```

The important rule is:

> **Patterns should emerge from real design problems. A pattern is successful when it makes the system simpler to change and understand—not merely when developers can identify the pattern by name.**

PART 16 — QUICK REVISION

Pattern	Category	Core purpose
Singleton	Creational	One shared instance
Factory Method	Creational	Encapsulate creation
Abstract Factory	Creational	Create product family
Builder	Creational	Staged complex construction
Prototype	Creational	Clone configured object
Adapter	Structural	Convert interface
Bridge	Structural	Separate two varying dimensions
Composite	Structural	Treat leaf and group uniformly
Decorator	Structural	Add behavior by wrapping
Facade	Structural	Simplify subsystem
Flyweight	Structural	Share repeated state
Proxy	Structural	Control access
Chain of Responsibility	Behavioral	Processing pipeline
Command	Behavioral	Encapsulate request
Interpreter	Behavioral	Interpret grammar
Iterator	Behavioral	Traverse sequence
Mediator	Behavioral	Centralize communication
Memento	Behavioral	Save/restore state
Observer	Behavioral	Publish notifications
State	Behavioral	Behavior varies by state
Strategy	Behavioral	Swap algorithms
Template Method	Behavioral	Fixed algorithm skeleton
Visitor	Behavioral	Add operations to stable hierarchy

PART 17 — FINAL PATTERN-SELECTION FLOW

EXAMPLE / FLOW

```
Do I have a creation problem?
        ↓
Yes
        ↓
```

EXAMPLE / FLOW

```
Need one shared instance?
    →
Singleton
```

EXAMPLE / FLOW

```
Need runtime object creation?
    →
Factory
```

EXAMPLE / FLOW

```
Need related families?
    →
Abstract Factory
```

EXAMPLE / FLOW

```
Need complex staged construction?
    →
Builder
```

EXAMPLE / FLOW

```
Need cloning?
    →
Prototype
```

EXAMPLE / FLOW

```
Do I have a structural problem?
        ↓
Yes
        ↓
```

EXAMPLE / FLOW

```
Incompatible interface?
    →
Adapter
```

EXAMPLE / FLOW

```
Need wrapper behavior?
    →
Decorator
```

EXAMPLE / FLOW

```
Need access control?
    →
Proxy
```

EXAMPLE / FLOW

Complex subsystem?
→
Facade

EXAMPLE / FLOW

Tree structure?
→
Composite

EXAMPLE / FLOW

Two independent dimensions?
→
Bridge

EXAMPLE / FLOW

Huge repeated shared state?
→
Flyweight

EXAMPLE / FLOW

Do I have a behavioral problem?
↓
Yes
↓

EXAMPLE / FLOW

Different algorithms?
→
Strategy

EXAMPLE / FLOW

Behavior changes with internal state?
→
State

EXAMPLE / FLOW

One-to-many notification?
→
Observer

EXAMPLE / FLOW

Pipeline of handlers?
→
Chain of Responsibility

EXAMPLE / FLOW

Represent action as object?
→
Command

EXAMPLE / FLOW

```
Centralize communication?
    →
Mediator
```

EXAMPLE / FLOW

```
Fixed algorithm with replaceable steps?
    →
Template Method
```

EXAMPLE / FLOW

```
Sequence traversal?
    →
Iterator
```

EXAMPLE / FLOW

```
Need undo/state snapshot?
    →
Memento
```

EXAMPLE / FLOW

```
Add operations over stable hierarchy?
    →
Visitor
```

EXAMPLE / FLOW

```
Domain-specific language?
    →
Interpreter
```

PART 18 — FINAL END-TO-END CONCEPT FLOW

EXAMPLE / FLOW

```
Business Requirement
        ↓
Understand actual design problem
        ↓
Identify variation/coupling
        ↓
Ask whether simple code is enough
```

If yes:

EXAMPLE / FLOW

```
Use simple code
```

If no:

EXAMPLE / FLOW

```
Choose appropriate pattern
        ↓
Keep responsibilities explicit
        ↓
Prefer composition
        ↓
Integrate using DI
```

```
        ↓
Keep lifetimes correct
        ↓
Test behavior through contracts
        ↓
Measure performance
        ↓
Review security/thread safety
```

For your Fundraising Platform:

EXAMPLE / FLOW

```
HTTP Request
    ↓
ASP.NET Core Middleware
    →
Chain of Responsibility
    ↓
Controller
    ↓
Gift Application Service
    →
Facade / Service Layer
    ↓
Validation Pipeline
    →
Chain of Responsibility
    ↓
Payment Behavior
    →
Strategy
    ↓
External Payment Provider
    →
Adapter
    ↓
Gift Domain
    →
Encapsulation / possibly State
    ↓
Repository
    →
Persistence abstraction
    ↓
DbContext
    →
Unit-of-Work concepts
    ↓
Receipt Sender
    →
Strategy
    ↓
Audit / Metrics wrappers
    →
Decorator
    ↓
Gift Completion notifications
    →
Observer
```

The most important final principle is:

A design pattern is not a goal. Maintainable software is the goal. Use Factory when creation is the real problem, Strategy when algorithms vary, Adapter when interfaces conflict, Decorator when behavior needs layering, Observer when notification is appropriate, State when behavior genuinely varies by lifecycle state, and simple direct code when none of those problems actually exist.

Next roadmap topic: Topic 34 — .NET Application Architecture.